

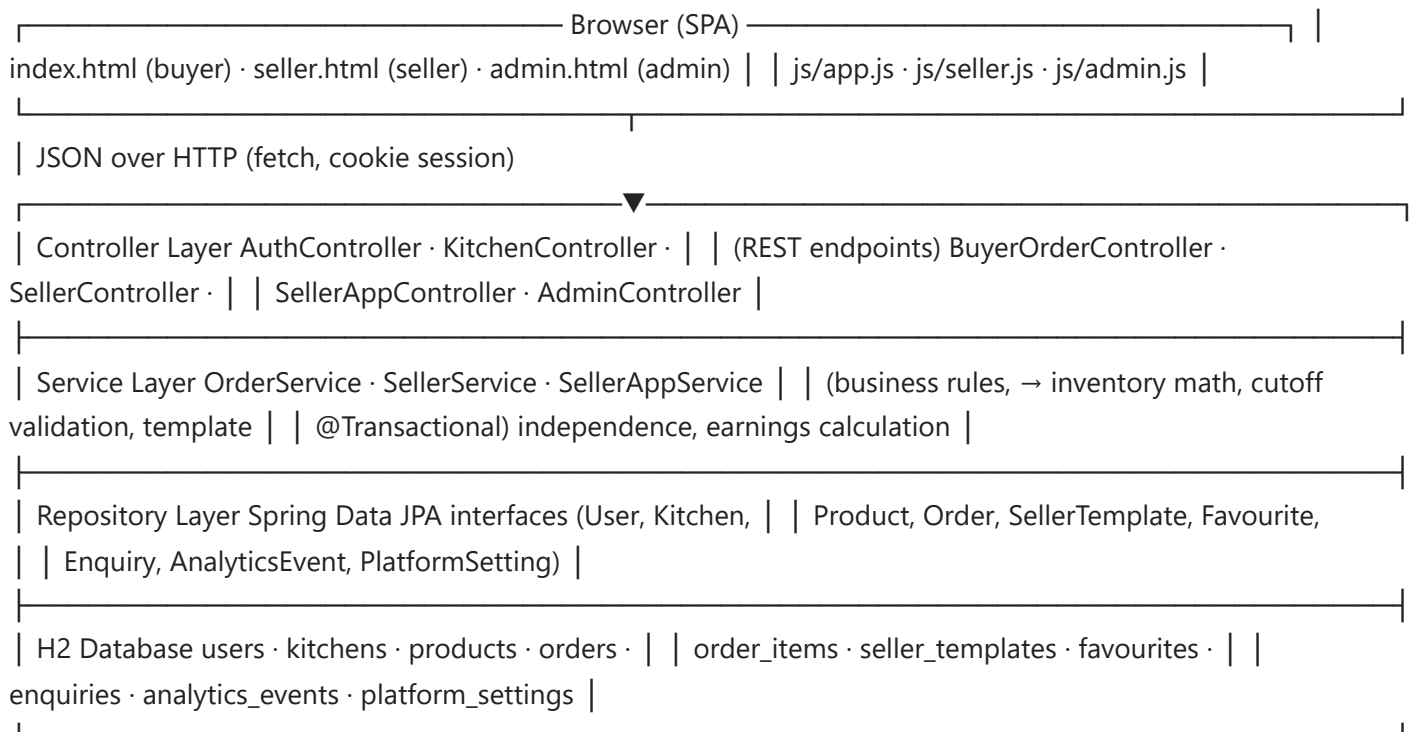
SocioMart Technical Document

1. Technology Stack

| Layer | Technology | Why It Was Chosen | |---|---|---| | Language | Java 21 | Stable LTS, strong typing, industry standard for enterprise APIs | | Framework | Spring Boot 4.1.1 | Fast REST development, built-in dependency injection, security, validation | | Data Layer | Spring Data JPA + Hibernate | Object-relational mapping without boilerplate SQL; automatic schema from entities | | Database | H2 (in-memory by default) | Relational integrity for orders/payments; reseeded with Indian food demo data on every boot | | Build Tool | Maven (mvnw wrapper) | Reproducible builds — no local install needed, works in CI | | Frontend | HTML5 + CSS3 + Vanilla JS (SPA) | Zero build step, instant load, served straight from Spring's static folder | | Sessions | HttpSession cookies | Simple, secure login state for buyer/seller flows | | Public Access | Render (Docker, free tier) | Free HTTPS public URL — single service serves Buyer (/) and Seller (/seller.html) with shared H2 state | | CI | GitHub Actions | Every push is auto-compiled so broken code can never reach main | | Docs & Testing | Edge headless (PDF), PowerShell E2E harness | Screenshot-driven docs, regression suite |

2. System Architecture

The application follows a classic 3-layer architecture. The browser never talks to the database directly — every request flows through the API layer, which enforces login and ownership rules. ``



Key design decision: `GlobalExceptionHandler` converts every business error (sold-out item, unauthorised seller, cap reached) into a clean JSON `{error, message}` response with the correct HTTP status — the frontend never crashes on an unexpected payload.

2.1 Buyer Route Ownership

The buyer SPA uses hash-based routing:

- `/#/kitchens` → `kitchensView()` — the dedicated client-facing Buyer route
- `/#/kitchen/{id}` → `kitchenPageView()` — public kitchen detail
- `/#/home`, `/#/food`, `/#/summary`, `/#/confirm`, `/#/payment`, `/#/favourites`, `/#/orders`, `/#/profile`

The raw `/kitchens` path is publicly accessible but has backend/admin route ownership conflict and is not the final Buyer navigation path.

2.2 Frontend CTA Fix

`updateCartBar()` in `static/js/common.js` hides `#viewOrderBar` when:

- cart has zero items
- OR
- current view contains `.sticky-footer-bar`

This prevents duplicate/overlapping Place Order CTAs on Order Summary, Confirm Order, and Payment screens.

3. Database Design

Eight tables, one per real-world concept. Relationships use JPA `ManyToOne` joins; demo data is seeded by `DemoDataSeeder` on first boot (idempotent, guarded by `platform_settings` flags).

Table	Purpose	Key Columns
users	Buyers, sellers, admin	mobile_number (unique), name, role, is_seller, flat_number, society, building
kitchens	A seller's shopfront	seller_id (FK), name, slug, rating, available_today, verified, society, building, image_url, instagram_link
products	One day's live offering	kitchen_id (FK), price, max_quantity, remaining_quantity, available_date, cutoff_time, ready_by_time, is_preorder, preorder_type
orders	One purchase event	buyer_id, kitchen_id, total_amount, payment_status, order_status, buyer_name, buyer_flat, order_number
order_items	Line items of an order	order_id (FK), product_id (FK), product_name, quantity, unit_price
seller_templates	Saved favourites (max 3)	seller_id (FK), name, description, price, max_quantity, cutoff_time, order_window_start, order_window_end
favourites	Buyer favourite kitchens	user_id (FK), kitchen_id (FK)
enquiries	Customer inquiries	user_id (FK), kitchen_id (FK), message, status, created_at
analytics_events	Raw behaviour log	kitchen_id, event_type (MENU_VIEW...), created_at
platform_settings	Seed guards and config	setting_key (unique), setting_value

Why remaining_quantity is its own column: the stepper updates one integer per click instead of re-deriving stock from orders — fast and race-safe. Why order_items.product_name is denormalised: a dish can be renamed or deleted later; the historical order must still show exactly what the buyer ordered.

4. REST API Reference

All endpoints return JSON. Errors always arrive as `{ "error": " ", "message": " " }` with a proper HTTP status (400 bad input, 401 not logged in, 403 wrong owner, 404 missing, 409 conflict).

4.1 Auth & Public

Method & Path	Who	What It Does
POST /api/auth/otp/request	public	Request OTP for mobile number
POST /api/auth/otp/verify	public	Verify OTP, create session
POST /api/auth/logout	authenticated	Invalidate session
GET /api/auth/me	authenticated	Current user info
GET /api/kitchens	public	Public kitchen list
GET /api/kitchens/id/{id}	public	One kitchen with today's offerings (logs MENU_VIEW)
GET /api/discovery/kitchens	public	Kitchen discovery with tab filters
GET /api/discovery/items	public	Item discovery by category
GET /api/discovery/category-kitchens	public	Kitchens by category
GET /api/discovery/counts	public	Kitchen counts by tab
GET /api/marketplace	public	Marketplace data
GET /api/items	public	Items list
GET /api/search	public	Search

4.2 Buyer

Method & Path	Who	What It Does
POST /api/buyer/orders/draft	buyer	Price the cart server-side
POST /api/buyer/orders/place	buyer	Place order, decrement stock atomically
GET /api/buyer/orders/my	buyer	Buyer's own orders
GET /api/buyer/orders/{id}	buyer	Buyer's own order detail
PUT /api/buyer/profile	buyer	Update buyer profile
GET /api/favourites	authenticated	List user's favourite kitchens
POST /api/favourites/kitchen/{id}/toggle	authenticated	Toggle favourite kitchen (max 3)
POST /api/enquiries	authenticated	Submit enquiry

4.3 Seller

Method & Path	Who	What It Does
GET /api/seller-app/dashboard	seller	Earnings, offerings, orders, real view count
GET /api/seller-app/templates	seller	List seller's favourite templates
POST /api/seller-app/templates	seller	Save offering as template (max 3)
DELETE /api/seller-app/templates/{id}	seller	Delete template
POST /api/seller-app/templates/{id}/publish	seller	Republish template as independent offering
POST /api/seller-app/products	seller	Create offering (manual / quick-post confirm)
PUT /api/seller/products/{id}	seller	Partial update (price, cutoff, stock...)

Method	Path	Who	What It Does
PATCH	/api/seller-app/products/{id}/inventory	seller	Stepper: add/subtract stock by delta
POST	/api/seller-app/products/{id}/sold-out	seller	Force mark sold out (works on unlimited items)
GET	/api/seller-app/orders/summary	seller	Per-dish plates + revenue
GET	/api/seller-app/orders/product/{id}	seller	Per-buyer drill-down
GET	/api/seller-app/earnings	seller	Pending / confirmed-today / this-month
GET	/api/seller-app/history	seller	Recent items for republish
POST	/api/seller-app/parse-message	seller	WhatsApp Quick Post parser (read-only)
PATCH	/api/seller/orders/{id}/status	seller	Order pipeline transitions (incl. cancel)
GET	/api/seller/kitchen	seller	Seller's kitchen details
PUT	/api/seller/kitchen	seller	Update kitchen profile

4.4 Admin

Method & Path	Who	What It Does
GET /api/admin/dashboard	admin	Dashboard stats
GET /api/admin/buyers	admin	List all buyers
GET /api/admin/sellers	admin	List all sellers
GET /api/admin/sellers/pending	admin	List pending sellers
POST /api/admin/sellers/{id}/approve	admin	Approve seller
POST /api/admin/sellers/{id}/reject	admin	Reject seller
GET /api/admin/kitchens	admin	List all kitchens
GET /api/admin/products	admin	List all offerings
GET /api/admin/orders	admin	List all orders
GET /api/admin/enquiries	admin	List all enquiries
GET /api/admin/analytics	admin	Platform analytics

5. Buyer Favourite Frontend Implementation

- Heart buttons use `data-action="toggle-fav-kitchen"` with `data-kid` for kitchen ID
- `toggleFavourite(type, id, btnEl)` in `app.js` calls `POST /api/favourites/{type}/{id}/toggle`
- `FAV\_CACHE` (client-side `Set`) stores favourited kitchen IDs and is kept in sync with backend responses
- `loadFavSet()` loads favourites from `/api/favourites` on every navigation to ensure fresh state
- On successful add/remove, the cache is updated, the button class/text/aria attributes are toggled, and a toast is shown
- On the Favourites screen, the list re-renders immediately after a toggle
- Duplicate rapid clicks are safe because the backend enforces uniqueness and returns the current state

6. Security Model

- Role-based access: BUYER, SELLER, ADMIN, SUPER_ADMIN
 - Buyer endpoints: require BUYER role
 - Seller endpoints: require SELLER role
 - Admin endpoints: require ADMIN or SUPER_ADMIN role
 - Super Admin endpoints: require SUPER_ADMIN role
 - Ownership checks on every mutation: seller B cannot modify seller A's kitchen/products/orders (HTTP 403)
 - ID isolation: buyer cannot access another buyer's order (HTTP 404)
 - Anonymous access to admin endpoints: rejected (HTTP 401)
 - Session timeout: 30 minutes
 - SameSite=Lax cookie protection
 - Custom UserSessionAuthorizationFilter runs before UsernamePasswordAuthenticationFilter
- Verified boundaries:
- anonymous → Admin rejected
 - Buyer → Admin rejected
 - Seller → Admin rejected
 - Buyer → seller orders rejected
 - Buyer → another buyer's order rejected
 - Seller cross-owner access rejected

7. Order Price Immutability

When an order is created, the current offering price is snapshotted into the order line item at transaction time:

- ``OrderItem.price`` stores the unit price at order creation
- ``Order.totalAmount`` is calculated from the snapshotted ``OrderItem.price`` values
- Changing a ``Product.price`` (catalogue price) does NOT cascade to existing ``OrderItem`` or ``Order`` records
- No JPA cascade, entity listener, database trigger, or frontend recalculation modifies historical order prices

Read paths that use persisted snapshots:

- Buyer order history: ``order.totalAmount` + `orderItem.price``
- Buyer order detail: ``order.totalAmount` + `orderItem.price``
- Seller order list: ``order.totalAmount``
- Seller order drill-down: ``orderItem.price` + `orderItem.quantity``
- Seller earnings: ``order.totalAmount` + `orderItem.price``
- Admin order list: ``order.totalAmount` + `orderItem.price``

- Admin GMV/order value: ``sum(order.totalAmount)`` – never recalculated from current ``Product.price``

New orders always read the current ``Product.price`` at draft/order creation time.

9. Testing

6.1 Unit Tests (JUnit 5)

13 tests across 3 test classes:

- **OrderServicePaymentTest** (5 tests): payment status → order status mapping
- **FavouriteServiceLimitTest** (4 tests): 3-kitchen favourite cap enforcement
- **OrderServiceValidationTest** (4 tests): cutoff, stock, duplicate order validation

6.2 Build Verification

- ``mvnw clean compile``: PASS (97 source files, Java 21)
- ``mvnw test``: 13/13 PASS
- ``mvnw package -DskipTests``: PASS

6.3 E2E Regression Suite

A PowerShell harness drives the real running application over HTTP – the same path a browser takes – and asserts behaviours. It is delta-based, so it can be re-run on a live instance any number of times without false failures.

10. Deployment

Hosting provider: Render.com – free Docker web service. The app is built from the existing Dockerfile (multi-stage Maven → JRE 21) and deployed as a Render Blueprint via `render.yaml`. One service serves both Buyer (/) and Seller (/seller.html) with a shared in-process H2 database – no split instances, no data desync. Render injects the `PORT` environment variable automatically; the app binds to ``${PORT:8081}``. Render auto-deploys on every push to main. The free tier sleeps after ~15 min of inactivity; a free uptime pinger keeps the service warm. Public URLs (single Render service):

- <https://sociomart-demo.onrender.com/> – Buyer view
- <https://sociomart-demo.onrender.com/seller.html> – Seller dashboard
- <https://sociomart-demo.onrender.com/admin.html> – Admin console
- <https://sociomart-demo.onrender.com/api/kitchens> – health probe

11. Known Limitations

- H2 in-memory database: custom data disappears on restart; demo data re-seeds on every boot (accepted V1/demo limitation)
- Free tier cold-start delay (~120s) on Render – expected demo behaviour
- No real OTP delivery (dev mode returns fixed OTP)
- Mockito self-attaching warning on JDK 21 (non-fatal)

12. UI/UX Fix (Latest)

Issue: Duplicate "Place Order" CTA on Order Summary, Confirm Order, and Payment screens – the cart bar (`#viewOrderBar`) overlapped with the sticky footer bar, creating two competing buttons. **Fix:** Modified `updateCartBar()` in `js/common.js` to detect screens with a `.sticky-footer-footer` element and hide the cart bar on those screens. The sticky footer already provides the CTA, so the cart bar is redundant.

Files changed: `my-first-spring-api/src/main/resources/static/js/common.js`